Taller práctico bloque 7

Del modelo de clases orientado a objetos a un diseño por tres capas para una API REST

Diseño de Software · Cuarto semestre · Caso de estudio: SmartLibrary

Integrantes: Esteban Santiago Narvaez Narvaez, Sebastian Granda Cordoba

Repositorio: https://github.com/Santix19/taller-practico- (el proyecto está en la carpeta taller-practico)

Servicio desplegado en Render: https://smartlibrary-0i60.onrender.com/api/prestamos

1. Diagrama inicial anotado (Actividad 1)

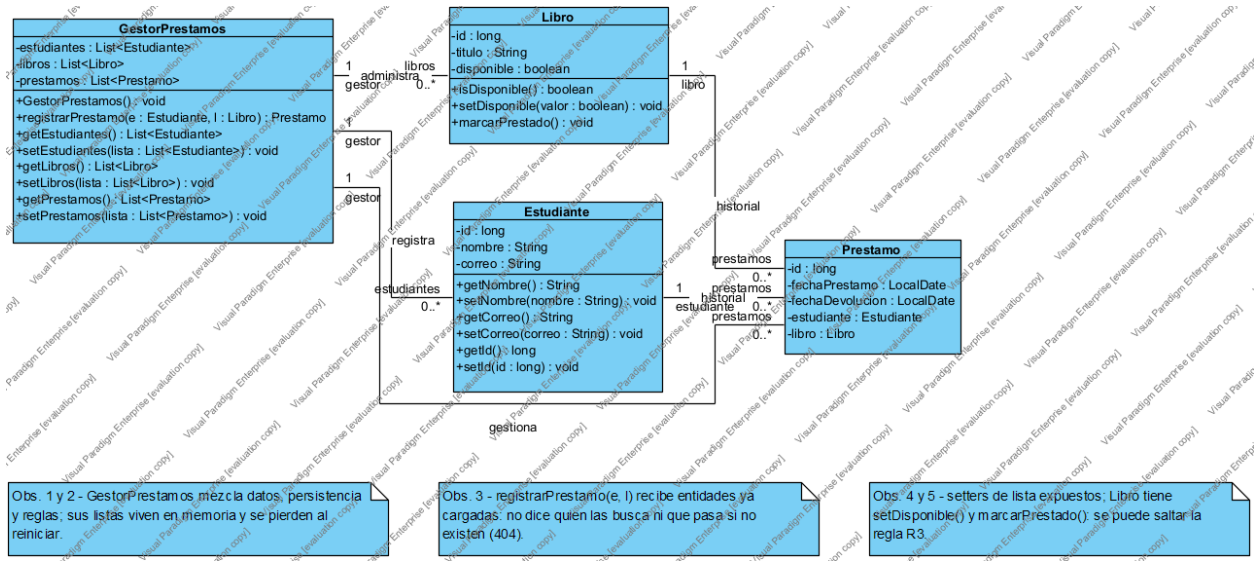


Figura 1. Modelo inicial del taller, dibujado y anotado en Visual Paradigm. Las notas «Obs.» corresponden a las observaciones numeradas de la tabla siguiente.

Observaciones al modelo inicial

N.º	Observación	Consecuencia / corrección
1	GestorPrestamos guarda tres listas (estudiantes, libros, préstamos), crea préstamos y coordina el caso de uso. Mezcla datos, persistencia en memoria y reglas.	Baja cohesión y alto acoplamiento. Se elimina: sus tareas pasan a PrestamoService (coordinar) y a los repositorios (guardar y buscar).
2	Las listas viven en memoria: al reiniciar la aplicación se pierde todo.	Se necesita persistencia real: EstudianteRepository, LibroRepository y PrestamoRepository (JPA con H2).
3	registrarPrestamo(e, l) recibe entidades ya cargadas. No dice quién las busca ni qué pasa si no	Falta buscar por id y lanzar 404 si no existen (R4).

	existen.	
4	Los setters de lista (setEstudiantes, setLibros, setPrestamos) exponen la estructura interna.	Se eliminan; el acceso a datos pasa por los repositorios.
5	Libro ofrece setDisponible() y marcarPrestado(): hay dos formas de cambiar el estado y se puede saltar R3.	Se conserva solo marcarPrestado() para prestar (y marcarDisponible() para la futura devolución).
6	Las asociaciones Gestor–Estudiante, Gestor–Libro y Gestor–Prestamo existen solo porque el gestor tiene colecciones.	Desaparecen con el gestor. Se conservan Prestamo→Estudiante y Prestamo→Libro.
7	Las líneas no tienen navegabilidad. Prestamo necesita llegar a Estudiante y Libro, pero estos no necesitan llegar a Prestamo.	En el rediseño las asociaciones son unidireccionales desde Prestamo.
8	No hay nada para recibir HTTP, validar la entrada ni traducir errores a códigos de estado.	Se agregan PrestamoController, PrestamoRequest, PrestamoResponse y GlobalExceptionHandler.

Respuestas de la Actividad 1

a) Datos y operaciones propios del dominio

- Estudiante: id, nombre, correo.
- Libro: id, titulo, disponible. Operaciones del dominio: isDisponible() y marcarPrestado() (R1 y R3).
- Prestamo: id, fechaPrestamo, fechaDevolucion [0..1], estudiante, libro (R2).
- Los getters y setters son de acceso. La coordinación del caso de uso y el almacenamiento no son del dominio.

b) Por qué Libro–Prestamo y Estudiante–Prestamo son asociaciones y no composiciones

Una composición exige que la parte no exista sin el todo y que viva y muera con él. Aquí no pasa: un libro o un estudiante existe antes de cualquier préstamo y sigue existiendo si el préstamo se elimina. Tampoco es agregación, porque un préstamo no es «un todo formado por» un libro y un estudiante. Es una relación de referencia: Prestamo guarda una referencia al estudiante y otra al libro. Por eso son asociaciones simples, con multiplicidad 0..* del lado de Prestamo y 1 del lado de Estudiante y Libro.

c) Operaciones necesarias

- Consultar: buscar estudiante por id, buscar libro por id, listar préstamos.
- Validar: que existan el estudiante y el libro; que el libro esté disponible (R1).
- Guardar: persistir el préstamo y el cambio de disponibilidad del libro, de forma atómica.

d) Responsabilidades que faltan

- Para recibir una petición HTTP: mapear POST y GET, leer y validar el cuerpo (PrestamoRequest), responder 201 o 200, y traducir las excepciones a 404, 409 y 400.

- Para acceder a datos: findById, save, findAll, buscar el libro con bloqueo para evitar préstamos simultáneos, y una transacción que cubra todo el caso de uso.

2. Tabla de responsabilidades (Actividad 2)

Operación o decisión	Clase responsable	Justificación
Recibir POST /api/prestamos	PrestamoController.registrar()	Es la única clase que conoce HTTP. Recibe el JSON, lo valida (@Valid) y delega. No contiene reglas de negocio.
Buscar estudiante por identificador	EstudianteRepository.findById() (lo invoca PrestamoService)	Acceder a datos es responsabilidad del repositorio. El servicio decide cuándo buscar; el repositorio sabe cómo.
Buscar libro por identificador	LibroRepository.buscarParaPrestar( ) (lo invoca PrestamoService)	Igual que el anterior, y además bloquea el libro hasta el final de la transacción para que dos solicitudes no presten el mismo libro (R5).
Validar disponibilidad	PrestamoService, consultando a Libro.isDisponible()	Libro conoce su estado (GRASP Experto). El servicio aplica la regla R1 porque es parte del caso de uso y lanza LibroNoDisponibleException.
Crear el préstamo	PrestamoService	GRASP Creador: el servicio tiene el estudiante, el libro y la fecha, que son los datos para inicializar el Prestamo (R2).
Cambiar disponibilidad	Libro.marcarPrestado()	El libro es el experto en su propio estado (R3). Así nadie más modifica el atributo directamente.
Guardar cambios	PrestamoRepository.save() y, al terminar la transacción, JPA actualiza Libro	La persistencia es del repositorio. La anotación @Transactional del servicio garantiza que el préstamo y el cambio del libro se guarden juntos o no se guarde nada (R4).
Devolver respuesta HTTP	PrestamoController (201 y 200) y GlobalExceptionHandler (404, 409, 400)	Elegir el código de estado y construir el cuerpo es salida HTTP, o sea, capa de presentación.

3. Diagrama de clases por capas (Actividad 3)

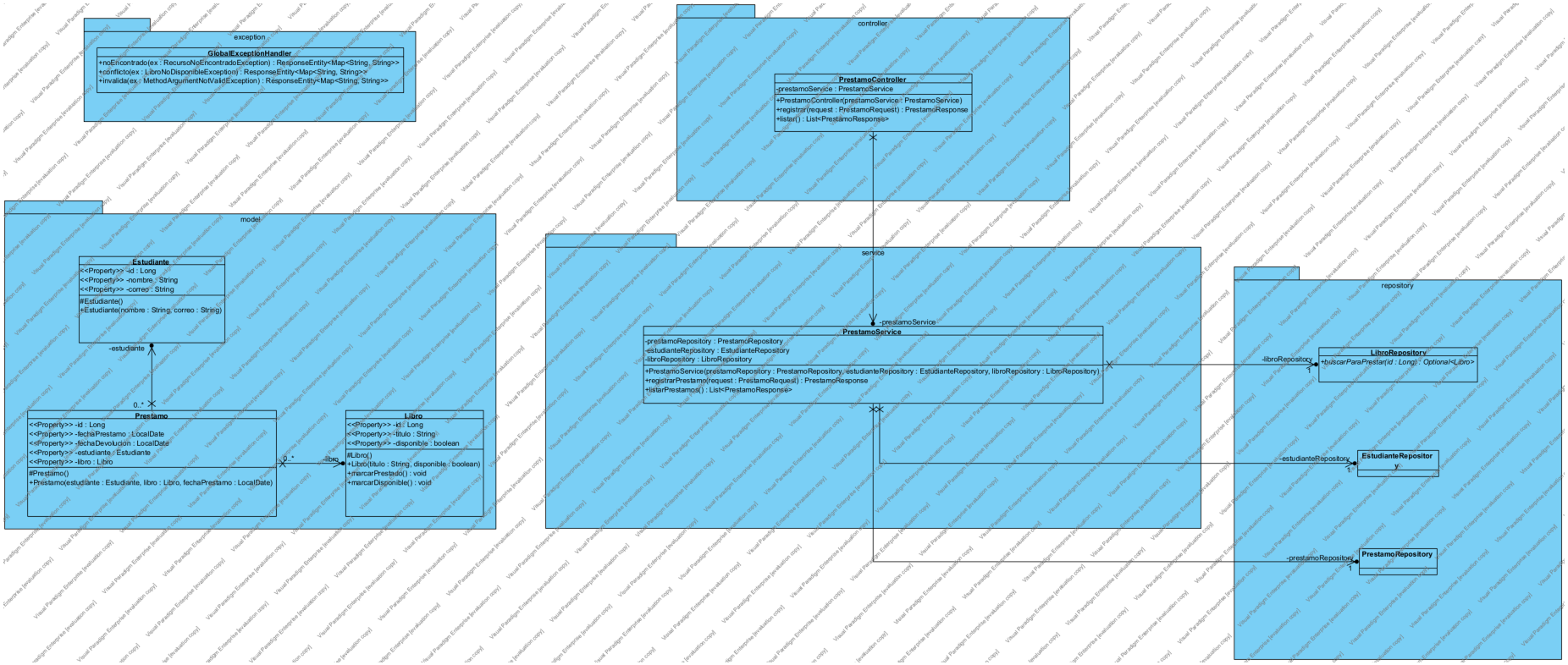


Figura 2. Diagrama de clases por capas hecho en Visual Paradigm, con un paquete por capa (controller, service, repository) más model y exception. Las líneas con flecha son asociaciones: el origen guarda una referencia en un atributo (Controller→Service, Service→cada Repository, Prestamo→Estudiante y Prestamo→Libro). Los DTO PrestamoRequest y PrestamoResponse son records de Java y se describen en la tabla de responsabilidades.

Justificación de relaciones y multiplicidades

Relación	Tipo	Multiplicidad	Justificación
PrestamoController → PrestamoService	Asociación (atributo)	1 a 1	El controlador mantiene una referencia inyectada al servicio durante toda su vida.
PrestamoService → los tres repositorios	Asociación (atributos)	1 a 1 cada una	El servicio guarda una referencia a cada repositorio que usa.
Controller / Service → PrestamoRequest, PrestamoResponse	Dependencia «use» / «create»	—	Solo se usan como parámetro o retorno de un método; no se guardan como atributo.
Repositorios → JpaRepository	Generalización entre interfaces (en el código)	—	En el código cada repositorio extiende JpaRepository y Spring Data genera la clase que lo implementa. En el diagrama solo se muestran las operaciones propias (p. ej. buscarParaPrestar) para no saturarlo.
Repositorios → entidades	Dependencia «manages»	—	El repositorio trabaja con la entidad como tipo genérico, sin guardarla como atributo.
Prestamo → Estudiante	Asociación unidireccional	Prestamo 0..* — 1 Estudiante	Un estudiante puede tener cero o muchos préstamos; cada préstamo es de exactamente un estudiante.
Prestamo → Libro	Asociación unidireccional	Prestamo 0..* — 1 Libro	Un libro puede tener muchos préstamos históricos (R5); cada préstamo es de un solo libro. Solo uno puede estar activo.

No se usan agregación ni composición: ninguna entidad es parte de otra con ciclo de vida dependiente. No hay flecha de Controller a Repository, así que se respeta la dirección Controller → Service → Repository.

4. Diagrama de secuencia (Actividad 4)

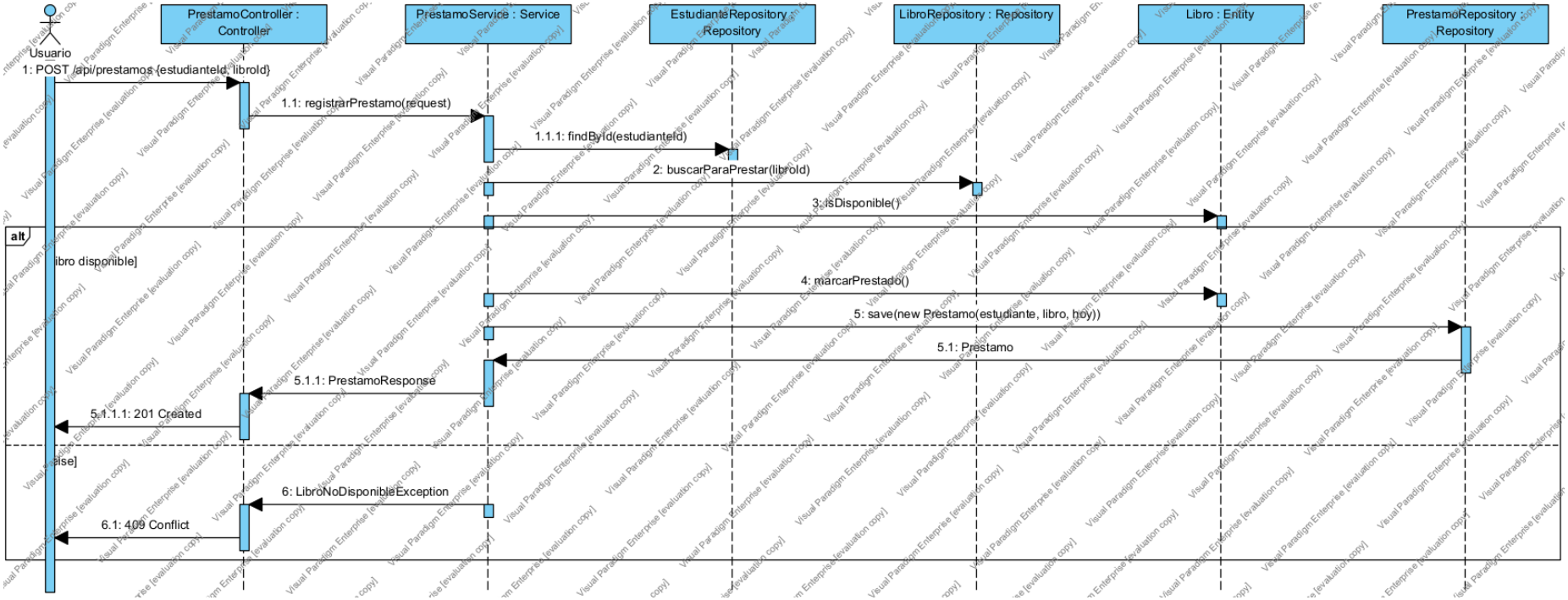


Figura 3. POST /api/prestamos con dos caminos: préstamo exitoso (201) y libro no disponible (409). Si el estudiante o el libro no existen, el servicio lanza RecursoNoEncontradoException y GlobalExceptionHandler responde 404 (no se dibuja para no saturar el diagrama). Diagrama hecho en Visual Paradigm.

Verificación mensaje del diagrama de secuencia → operación del diagrama de clases

Mensaje	Operación en el diagrama de clases
POST /api/prestamos	PrestamoController.registrar(request)
registrarPrestamo(request)	PrestamoService.registrarPrestamo(request: PrestamoRequest): PrestamoResponse
findById(estudianteId)	EstudianteRepository (heredada de JpaRepository.findById)
buscarParaPrestar(libroId)	LibroRepository.buscarParaPrestar(id)
isDisponible()	Libro.isDisponible()
marcarPrestado()	Libro.marcarPrestado()
save(new Prestamo(...))	PrestamoRepository (heredada de JpaRepository.save) y constructor de Prestamo
PrestamoResponse	PrestamoResponse.desde(p) (usado por el servicio)
LibroNoDisponibleException → 409; RecursoNoEncontradoException → 404	GlobalExceptionHandler (capa de presentación)

5. Proyecto Spring Boot y pruebas (Actividades 5 y 6)

El proyecto está en la raíz del repositorio. Usa Java 21, Maven, Spring Web, Spring Data JPA, Validation y H2. Paquetes: controller, service, repository, model, dto y exception (este último agrupa las excepciones y el manejador de errores).

Para ejecutar: mvn spring-boot:run · Para probar: mvn test

Escenario	Resultado esperado	Prueba (PrestamoApiTest)
Préstamo de libro disponible	201 Created y préstamo registrado	prestamoDeLibroDisponible_devuelve201
Libro ya prestado	409 Conflict; sin nuevo préstamo	libroYaPrestado_devuelve409_yNoCreaPrestamo
Estudiante inexistente	404 Not Found	estudianteInexistente_devuelve404
Libro inexistente	404 Not Found	libroInexistente_devuelve404
Consulta de préstamos	200 OK con registros	consultaDePrestamos_devuelve200ConRegistros
Solicitud sin campos (R4)	400 Bad Request	solicitudSinCampos_devuelve400

Evidencia de ejecución en el servicio desplegado en Render (12 de noviembre de 2026). Se enviaron solicitudes POST /api/prestamos con los datos iniciales:

Solicitud	Resultado obtenido	Regla
estudianteId 1, libroId 1 (disponible)	201 Created: préstamo registrado con fecha de hoy	R1, R2, R3
estudianteId 2, libroId 1 (ya prestado en la solicitud anterior)	409 Conflict: el libro 'Clean Code' no esta disponible	R3, R5
estudianteId 1, libroId 3 (prestado desde el inicio)	409 Conflict: el libro 'UML Distilled' no esta disponible	R1, R4
estudianteId 99, libroId 2	404 Not Found: Estudiante no	R4

	encontrado: 99	
estudianteId 1, libroId 99	404 Not Found: Libro no encontrado: 99	R4
Cuerpo vacío {}	400 Bad Request: libroId es obligatorio	R4
GET /api/prestamos	200 OK con los préstamos registrados (el inicial y el nuevo)	Consulta

Las mismas pruebas están automatizadas en PrestamoApiTest y se ejecutan con mvn test.

Decisiones que se reflejan en el UML

- Las entidades llevan «Entity» y siguen siendo modelo de dominio, no una cuarta capa.
- Se usan registros (record) como DTO para no exponer las entidades JPA en la API.
- El préstamo y el cambio de disponibilidad ocurren en una sola transacción (@Transactional en el servicio).
- LibroRepository busca el libro con bloqueo pesimista para evitar que dos solicitudes simultáneas presten el mismo libro.
- Los errores se traducen a HTTP en un único lugar: GlobalExceptionHandler.

6. Revisión y defensa (Actividad 7)

1) ¿Qué responsabilidades cambiaron del modelo inicial al modelo por capas?

- La coordinación del caso de uso salió de GestorPrestamos y pasó a PrestamoService.
- Guardar y buscar dejó de ser listas en memoria y pasó a tres repositorios.
- Recibir y responder HTTP pasó a PrestamoController, con DTO y un manejador de excepciones.
- El cambio de estado del libro quedó solo en Libro.marcarPrestado().
- GestorPrestamos desapareció.

2) ¿Qué patrón GRASP se reconoce en la coordinación del caso de uso? ¿Por qué un controlador GRASP no equivale necesariamente a @RestController?

Se reconoce el patrón Controlador de GRASP: asigna a un objeto la responsabilidad de recibir y coordinar una operación del sistema (aquí «registrar préstamo»). En este diseño esa coordinación la hace PrestamoService, que decide el orden de los pasos y delega el trabajo a los expertos. @RestController es una anotación de Spring de la capa web: sirve para adaptar HTTP (leer JSON, elegir el código de estado) y es un adaptador entre la interfaz y la lógica. El controlador GRASP es un rol de diseño y la anotación es un detalle del framework; una clase con @RestController que contuviera las reglas del préstamo sería un mal controlador GRASP y rompería la separación de capas. Otros patrones presentes: Experto (Libro.marcarPrestado), Creador (el servicio crea el Prestamo), Bajo acoplamiento y Alta cohesión (cada capa con una sola razón de cambio), Fabricación pura (los repositorios) e Indirección (el servicio entre el controlador y los datos).

3) ¿Dónde se ubica la regla de disponibilidad y por qué?

El dato y su cambio están en Libro (isDisponible, marcarPrestado), porque la entidad es la experta en su estado. La decisión de rechazar el préstamo (R1) está en PrestamoService, porque es parte del caso de uso y necesita coordinar varias entidades. No está en el Controller, que solo conoce HTTP, ni en el Repository, que solo guarda y busca.

4) ¿Qué relaciones UML se conservaron y cuáles se modificaron?

- Conservadas: asociaciones Prestamo–Estudiante y Prestamo–Libro, con 0..* del lado del préstamo y 1 del otro. Ahora son unidireccionales desde Prestamo. Sigue sin haber agregación ni composición.
- Eliminadas: las asociaciones de GestorPrestamos con las tres entidades.
- Nuevas: asociaciones Controller→Service y Service→Repository (1 a 1); uso de los DTO como parámetro y retorno; Prestamo→Estudiante y Prestamo→Libro (0..* a 1); en el código, cada repositorio extiende JpaRepository.

5) ¿Cómo se comprueba la trazabilidad requisito → clase → operación → prueba?

Requisito	Clase	Operación	Prueba
R1 Solo un libro disponible puede prestarse	PrestamoService, Libro	registrarPrestamo(), isDisponible()	libroYaPrestado_devuelve409_yNoCreaPrestamo
R2 El préstamo registra estudiante, libro y fecha	PrestamoService, Prestamo	registrarPrestamo(), new Prestamo(…), save()	prestamoDeLibroDisponible_devuelve201
R3 Al prestarlo, el libro deja de estar disponible	Libro	marcarPrestado()	prestamoDeLibroDisponible_devuelve201 (el segundo intento da 409)
R4 Una solicitud inválida no crea préstamo	PrestamoRequest, GlobalExceptionHandler, PrestamoService	@Valid, @Transactional	estudianteInexistente, libroInexistente, solicitudSinCampos
R5 Varios préstamos históricos, uno solo activo	Libro, LibroRepository	disponible, buscarParaPrestar()	libroYaPrestado_devuelve409_yNoCreaPrestamo
Consultar préstamos	PrestamoController, PrestamoService	listar(), listarPrestamos(), findAll()	consultaDePrestamos_devuelve200ConRegistros

7. Explicación de las decisiones de diseño (una página)

El modelo inicial concentraba todo en GestorPrestamos: guardaba colecciones en memoria, buscaba, validaba y creaba préstamos. Esa clase tenía baja cohesión y alto acoplamiento, y no ofrecía nada para recibir peticiones HTTP. Por eso la reemplazamos por un diseño de tres capas con una dirección de dependencia única: Controller → Service → Repository.

PrestamoController solo traduce HTTP: recibe un PrestamoRequest validado, delega en el servicio y devuelve 201 o 200. Un GlobalExceptionHandler convierte las excepciones de negocio en 404, 409 o 400. PrestamoService coordina el caso de uso «Registrar préstamo»: busca el estudiante y el libro, comprueba la disponibilidad (R1), cambia el estado del libro (R3) y crea el préstamo (R2). Lo hace dentro de una transacción, de modo que una solicitud inválida no deja cambios a medias (R4). Los repositorios, que Spring Data implementa, son la única puerta a los datos.

Las entidades Estudiante, Libro y Prestamo siguen siendo el modelo de dominio y no una cuarta capa. Libro es experto en su disponibilidad (marcarPrestado), siguiendo el patrón Experto. Mantuvimos las asociaciones Prestamo→Estudiante y Prestamo→Libro con multiplicidad 0..* a 1, porque un libro o estudiante existe con independencia de sus préstamos, así que no usamos composición ni agregación. Para R5, el libro se consulta con bloqueo dentro de la transacción: así solo un préstamo activo puede existir por libro aunque lleguen dos solicitudes a la vez.

Usamos DTO (PrestamoRequest y PrestamoResponse) para no exponer las entidades JPA. La trazabilidad de requisitos a pruebas está en la tabla de la Actividad 7: cada regla R1 a R5 tiene una clase, una operación y al menos una prueba automática que la verifica.